Q:- 1. Database Initialization and Table Creation: Create a New Database, and create tables: students table, courses table & enrollments table.

- **Input:-**
- **init_tables.py:-**

```
import sqlite3

def init_database():
    # 1. Connect to SQLite database (creates database file if it does not exist)
    conn = sqlite3.connect('university.db')
    cursor = conn.cursor()

    # Enable Foreign Key Support
    cursor.execute("PRAGMA foreign_keys = ON;")

    # 2. Create 'students' table
    cursor.execute('''
    CREATE TABLE IF NOT EXISTS students (
        student_id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        age INTEGER NOT NULL,
        major TEXT NOT NULL,
        email TEXT UNIQUE NOT NULL
    );
    ''')

    # 3. Create 'courses' table
    cursor.execute('''
    CREATE TABLE IF NOT EXISTS courses (
        course_id INTEGER PRIMARY KEY AUTOINCREMENT,
        course_code TEXT UNIQUE NOT NULL,
        course_name TEXT NOT NULL,
        credits INTEGER NOT NULL
    );
    ''')

    # 4. Create 'enrollments' table with Foreign Keys
    cursor.execute('''
    CREATE TABLE IF NOT EXISTS enrollments (
        enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
        student_id INTEGER NOT NULL,
        course_id INTEGER NOT NULL,
        grade TEXT,
```

```
        FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE
RESTRICT,
        FOREIGN KEY (course_id) REFERENCES courses(course_id) ON DELETE CASCADE
    );
    "")

    conn.commit()
    print("Database 'university.db' initialized successfully.")
    print("Tables created: students, courses, enrollments.")
    conn.close()

if __name__ == '__main__':
    init_database()
```

- **Output:-**

```
Database 'university.db' initialized successfully.
Tables created: students, courses, enrollments.
```

Q:- 2. Data Insertion (Create Operations): Insert Sample Data into students, courses, and enrollments tables.

- **Input:-**
- **insert_sample_data.py:-**

```
import sqlite3

def insert_data():
    conn = sqlite3.connect('university.db')
    cursor = conn.cursor()
    cursor.execute("PRAGMA foreign_keys = ON;")

    # Insert sample students
    students_data = [
        ('Amit Sharma', 20, 'Computer Science', 'amit.sharma@example.com'),
        ('Priya Patel', 21, 'Information Technology', 'priya.patel@example.com'),
        ('Rahul Verma', 22, 'Electronics', 'rahul.verma@example.com'),
        ('Sneha Joshi', 20, 'Computer Science', 'sneha.joshi@example.com')
    ]
    cursor.executemany('''
    INSERT OR IGNORE INTO students (name, age, major, email)
    VALUES (?, ?, ?, ?);
    ''', students_data)

    # Insert sample courses
    courses_data = [
        ('CS501', 'Database Systems', 4),
        ('CS504', 'Web Development Using MEAN Stack', 4),
        ('IT502', 'Cloud Computing', 3)
    ]
    cursor.executemany('''
    INSERT OR IGNORE INTO courses (course_code, course_name, credits)
    VALUES (?, ?, ?);
    ''', courses_data)

    # Insert sample enrollments
    enrollments_data = [
        (1, 1, 'A'),
        (1, 2, 'B'),
        (2, 2, 'A'),
        (3, 3, 'B'),
        (4, 1, 'A')
    ]
```

```
    cursor.executemany('''
    INSERT INTO enrollments (student_id, course_id, grade)
    VALUES (?, ?, ?);
    ''', enrollments_data)

    conn.commit()
    print("Sample data inserted successfully into students, courses, and enrollments.")
    conn.close()

if __name__ == '__main__':
    insert_data()
```

- **Output:-**

Sample data inserted successfully into students, courses, and enrollments.

Q:- 3. Data Retrieval (Read Operations): 1. Basic Retrieval, 2. Filtered Retrieval, 3. Complex Queries (using JOINs).

- **Input:-**
- **retrieve_data.py:-**

```
import sqlite3

def retrieve_data():
    conn = sqlite3.connect('university.db')
    cursor = conn.cursor()

    # 1. Basic Retrieval - All Students
    print("--- 1. Basic Retrieval (All Students) ---")
    cursor.execute("SELECT student_id, name, major, email FROM students;")
    for row in cursor.fetchall():
        print(f"ID: {row[0]} | Name: {row[1]} | Major: {row[2]} | Email: {row[3]}")

    # 2. Filtered Retrieval - Students with Computer Science Major
    print("\n--- 2. Filtered Retrieval (Major = 'Computer Science') ---")
    cursor.execute("SELECT name, email FROM students WHERE major = 'Computer
Science';")
    for row in cursor.fetchall():
        print(f"Student: {row[0]} | Email: {row[1]}")

    # 3. Complex Query - JOIN students, enrollments, and courses
    print("\n--- 3. Complex Queries (JOIN across Students, Enrollments, and Courses) ---")
    query = '''
    SELECT s.name, c.course_code, c.course_name, e.grade
    FROM enrollments e
    JOIN students s ON e.student_id = s.student_id
    JOIN courses c ON e.course_id = c.course_id
    ORDER BY s.name;
    '''
    cursor.execute(query)
    for row in cursor.fetchall():
        print(f"Student: {row[0]:<14} | Course: {row[1]} ({row[2]}) | Grade: {row[3]}")

    conn.close()

if __name__ == '__main__':
    retrieve_data()
```

➢ Output:-

```
--- 1. Basic Retrieval (All Students) ---
ID: 1 | Name: Amit Sharma | Major: Computer Science | Email: amit.sharma@example.com
ID: 2 | Name: Priya Patel | Major: Information Technology | Email: priya.patel@example.com
ID: 3 | Name: Rahul Verma | Major: Electronics | Email: rahul.verma@example.com
ID: 4 | Name: Sneha Joshi | Major: Computer Science | Email: sneha.joshi@example.com

--- 2. Filtered Retrieval (Major = 'Computer Science') ---
Student: Amit Sharma | Email: amit.sharma@example.com
Student: Sneha Joshi | Email: sneha.joshi@example.com

--- 3. Complex Queries (JOIN across Students, Enrollments, and Courses) ---
Student: Amit Sharma   | Course: CS501 (Database Systems) | Grade: A
Student: Amit Sharma   | Course: CS504 (Web Development Using MEAN Stack) | Grade: B
Student: Priya Patel   | Course: CS504 (Web Development Using MEAN Stack) | Grade: A
Student: Rahul Verma   | Course: IT502 (Cloud Computing) | Grade: B
Student: Sneha Joshi   | Course: CS501 (Database Systems) | Grade: A
```

Q:- 4. Data Update (Update Operations): 1. Update Student Major, 2. Update Enrollment Grade.

- **Input:-**
- **update_data.py:-**

```
import sqlite3

def update_operations():
    conn = sqlite3.connect('university.db')
    cursor = conn.cursor()

    # 1. Update Student Major
    cursor.execute('''
    UPDATE students
    SET major = 'Data Science'
    WHERE name = 'Amit Sharma';
    ''')
    print("1. Student Major Updated: Amit Sharma changed to Data Science.")

    # 2. Update Enrollment Grade
    cursor.execute('''
    UPDATE enrollments
    SET grade = 'A+'
    WHERE student_id = 1 AND course_id = 2;
    ''')
    print("2. Enrollment Grade Updated: Student ID 1 for Course ID 2 updated to A+.")

    conn.commit()

    # Verification Query
    print("\nVerification of Updated Records:")
    cursor.execute("SELECT name, major FROM students WHERE name = 'Amit Sharma';")
    print("Updated Student:", cursor.fetchone())

    cursor.execute("SELECT student_id, course_id, grade FROM enrollments WHERE
student_id = 1 AND course_id = 2;")
    print("Updated Enrollment:", cursor.fetchone())

    conn.close()

if __name__ == '__main__':
    update_operations()
```

➢ Output:-

1. Student Major Updated: Amit Sharma changed to Data Science.
2. Enrollment Grade Updated: Student ID 1 for Course ID 2 updated to A+.

Verification of Updated Records:
Updated Student: ('Amit Sharma', 'Data Science')
Updated Enrollment: (1, 2)

Q:- 5. Data Deletion (Delete Operations): 1. Delete an Enrollment, 2. Attempt to Delete Enrolled Student (Observe FK Constraint), 3. Safe Student Deletion.

- **Input:-**
- **delete_operations.py:-**

```
import sqlite3

def delete_operations():
    conn = sqlite3.connect('university.db')
    cursor = conn.cursor()
    cursor.execute("PRAGMA foreign_keys = ON;")

    # 1. Delete an Enrollment
    cursor.execute("DELETE FROM enrollments WHERE enrollment_id = 1;")
    conn.commit()
    print("1. Enrollment with ID 1 deleted successfully.")

    # 2. Attempt to Delete Enrolled Student (Observe FK Constraint)
    print("\n2. Attempting to delete student ID 2 who has active enrollments...")
    try:
        cursor.execute("DELETE FROM students WHERE student_id = 2;")
        conn.commit()
    except sqlite3.IntegrityError as e:
        print(f"FOREIGN KEY CONSTRAINT CAUGHT: {e}")
        print("Observation: The database rejected deletion because RESTRICT constraint
protects enrolled students.")

    # 3. Safe Student Deletion: Delete student's enrollments first, then student
    print("\n3. Executing Safe Student Deletion for Student ID 3:")
    cursor.execute("DELETE FROM enrollments WHERE student_id = 3;")
    cursor.execute("DELETE FROM students WHERE student_id = 3;")
    conn.commit()
    print("Enrollments for student 3 removed, followed by safe student deletion.")

    conn.close()

if __name__ == '__main__':
    delete_operations()
```

- **Output:-**

```
1. Enrollment with ID 1 deleted successfully.
```

2. Attempting to delete student ID 2 who has active enrollments...
FOREIGN KEY CONSTRAINT CAUGHT: FOREIGN KEY constraint failed
Observation: The database rejected deletion because RESTRICT constraint protects enrolled students.

3. Executing Safe Student Deletion for Student ID 3:
Enrollments for student 3 removed, followed by safe student deletion.

Q:- 6. Database Dumping: 1. Full Database Dump, 2. Schema-Only Dump, 3. Data-Only Dump.

- **Input:-**
- **database_dumps.py:-**

```
import sqlite3

def dump_database():
    conn = sqlite3.connect('university.db')

    # 1. Full Database Dump (Schema + Data)
    with open('full_dump.sql', 'w') as f:
        for line in conn.iterdump():
            f.write(f"{line}\n")
    print("1. Full Database Dump exported to 'full_dump.sql'.")

    # 2. Schema-Only Dump
    cursor = conn.cursor()
    cursor.execute("SELECT sql FROM sqlite_master WHERE type='table' AND name NOT
LIKE 'sqlite_%';")
    schema_statements = cursor.fetchall()
    with open('schema_dump.sql', 'w') as f:
        for stmt in schema_statements:
            if stmt[0]:
                f.write(f"{stmt[0]};\n\n")
    print("2. Schema-Only Dump exported to 'schema_dump.sql'.")

    # 3. Data-Only Dump (INSERT statements)
    with open('data_dump.sql', 'w') as f:
        for line in conn.iterdump():
            if line.startswith('INSERT INTO'):
                f.write(f"{line}\n")
    print("3. Data-Only Dump exported to 'data_dump.sql'.")

    conn.close()

if __name__ == '__main__':
    dump_database()
```

- **Output:-**

```
1. Full Database Dump exported to 'full_dump.sql'.
2. Schema-Only Dump exported to 'schema_dump.sql'.
3. Data-Only Dump exported to 'data_dump.sql'.
```

Q:- 7. CSV File Handling: 1. Export Enrollments to CSV, 2. Import New Students from CSV.

- **Input:-**
- **csv_handling.py:-**

```
import sqlite3
import csv

def handle_csv():
    conn = sqlite3.connect('university.db')
    cursor = conn.cursor()

    # 1. Export Enrollments to CSV
    cursor.execute('''
    SELECT e.enrollment_id, s.name, c.course_code, e.grade
    FROM enrollments e
    JOIN students s ON e.student_id = s.student_id
    JOIN courses c ON e.course_id = c.course_id;
    ''')
    rows = cursor.fetchall()
    with open('enrollments_export.csv', 'w', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(['Enrollment_ID', 'Student_Name', 'Course_Code', 'Grade'])
        writer.writerows(rows)
    print("1. Enrollments exported successfully to 'enrollments_export.csv'.")

    # 2. Import New Students from CSV
    # Create sample import CSV
    new_students = [
        ['Vikram Malhotra', 21, 'Cyber Security', 'vikram.m@example.com'],
        ['Ananya Sen', 20, 'Data Science', 'ananya.s@example.com']
    ]
    with open('new_students.csv', 'w', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(['name', 'age', 'major', 'email'])
        writer.writerows(new_students)

    with open('new_students.csv', 'r') as f:
        reader = csv.DictReader(f)
        for row in reader:
            cursor.execute('''
            INSERT OR IGNORE INTO students (name, age, major, email)
            VALUES (?, ?, ?, ?);
```

```
            ''', (row['name'], row['age'], row['major'], row['email']))

    conn.commit()
    print("2. New students imported successfully from 'new_students.csv'.")

    # Verification
    cursor.execute("SELECT student_id, name, major FROM students WHERE major IN
('Cyber Security', 'Data Science');")
    print("\nNewly Imported Students Verified:")
    for r in cursor.fetchall():
        print(f"ID: {r[0]} | Name: {r[1]} | Major: {r[2]}")

    conn.close()

if __name__ == '__main__':
    handle_csv()
```

- **Output:-**

```
1. Enrollments exported successfully to 'enrollments_export.csv'.
2. New students imported successfully from 'new_students.csv'.

Newly Imported Students Verified:
ID: 1 | Name: Amit Sharma | Major: Data Science
ID: 5 | Name: Vikram Malhotra | Major: Cyber Security
ID: 6 | Name: Ananya Sen | Major: Data Science
```

Journal Assignment – 3

Q:- 1. Setup Your Database Connection: 1. Import Module, 2. Connect to Database, 3. Create a Cursor.

- **Input:-**
- **step1_setup_connection.py:-**

```
# 1. Import Module
import sqlite3

# 2. Connect to Database
connection = sqlite3.connect('store.db')
print("Successfully connected to 'store.db'.")

# 3. Create a Cursor
cursor = connection.cursor()
print("Cursor created successfully to execute SQL statements.")

# Close connection
connection.close()
print("Connection closed.")
```

- **Output:-**

```
Successfully connected to 'store.db'.
Cursor created successfully to execute SQL statements.
Connection closed
```

Q:- 2. Create Your First Table (products): 1. Define SQL, 2. Execute SQL, 3. Save Changes, 4. Close Connection.

- **Input:-**
- **step2_create_table.py:-**

```
import sqlite3

connection = sqlite3.connect('store.db')
cursor = connection.cursor()

# 1. Define SQL
create_table_sql = '''
CREATE TABLE IF NOT EXISTS products (
    product_id INTEGER PRIMARY KEY AUTOINCREMENT,
    product_name TEXT NOT NULL,
    price REAL NOT NULL,
    quantity INTEGER NOT NULL
);
'''

# 2. Execute SQL
cursor.execute(create_table_sql)
print("Table 'products' created successfully.")

# 3. Save Changes
connection.commit()
print("Changes saved (committed).")

# 4. Close Connection
connection.close()
print("Connection closed.")
```

- **Output:-**

```
Table 'products' created successfully.
Changes saved (committed).
Connection closed.
```

Q:- 3. Add New Products (Create Operation - INSERT): 1. Re-connect and Get Cursor, 2. Insert Single Product, 3. Insert Multiple Products, 4. Confirm Insertions, 5. Commit and Close.

- **Input:-**
- **step3_insert_products.py:-**

```
import sqlite3
# 1. Re-connect and Get Cursor
conn = sqlite3.connect('store.db')
cur = conn.cursor(
# 2. Insert Single Product
cur.execute('''
INSERT INTO products (product_name, price, quantity)
VALUES (?, ?, ?);
''', ('Wireless Mouse', 450.00, 25))
print("Single product inserted: Wireless Mouse.")
 # 3. Insert Multiple Products
products_list = [
    ('Mechanical Keyboard', 2200.00, 15),
    ('HD Webcam', 1800.00, 5),
    ('USB-C Hub', 1250.00, 4),
    ('Gaming Headset', 2800.00, 18)
]
cur.executemany('''
INSERT INTO products (product_name, price, quantity)
VALUES (?, ?, ?);
''', products_list)
print("Multiple products inserted successfully.")

# 4. Confirm Insertions
cur.execute("SELECT COUNT(*) FROM products;")
total_count = cur.fetchone()[0]
print(f"Total products now in table: {total_count}")
# 5. Commit and Close
conn.commit()
conn.close()
print("Transaction committed and connection closed.")
```

- **Output:-**

```
Single product inserted: Wireless Mouse.
Multiple products inserted successfully.
Total products now in table: 5
Transaction committed and connection closed.
```

Q:- 4. View Your Products (Read Operation - SELECT): 1. Print Products, 2. Select Specific Product, 3. Select Products with Low Quantity.

- **Input:-**
- **step4_view_products.py:-**

```
import sqlite3

conn = sqlite3.connect('store.db')
cur = conn.cursor()

# 1. Print All Products
print("--- 1. All Products in Inventory ---")
cur.execute("SELECT * FROM products;")
for row in cur.fetchall():
    print(f"ID: {row[0]:2d} | Name: {row[1]:<20} | Price: Rs.{row[2]:7.2f} | Quantity: {row[3]}"
# 2. Select Specific Product by Name
print("\n--- 2. Select Specific Product ('Mechanical Keyboard') ---")
cur.execute("SELECT * FROM products WHERE product_name = ?;", ('Mechanical
Keyboard',))
prod = cur.fetchone()
print(f"Found: ID: {prod[0]} | Name: {prod[1]} | Price: Rs.{prod[2]} | Stock: {prod[3]}")

# 3. Select Products with Low Quantity (Quantity < 10)
print("\n--- 3. Products with Low Stock (Quantity < 10) ---")
cur.execute("SELECT product_name, quantity FROM products WHERE quantity < 10;")
for p in cur.fetchall():
    print(f"Low Stock Alert: {p[0]} (Only {p[1]} units left!)")

conn.close()
```

- **Output:-**

```
--- 1. All Products in Inventory ---
ID:  1 | Name: Wireless Mouse       | Price: Rs. 450.00 | Quantity: 25
ID:  2 | Name: Mechanical Keyboard  | Price: Rs.2200.00 | Quantity: 15
ID:  3 | Name: HD Webcam            | Price: Rs.1800.00 | Quantity: 5
ID:  4 | Name: USB-C Hub            | Price: Rs.1250.00 | Quantity: 4
ID:  5 | Name: Gaming Headset       | Price: Rs.2800.00 | Quantity: 18
--- 2. Select Specific Product ('Mechanical Keyboard') ---
Found: ID: 2 | Name: Mechanical Keyboard | Price: Rs.2200.0 | Stock: 15

--- 3. Products with Low Stock (Quantity < 10) ---
Low Stock Alert: HD Webcam (Only 5 units left!)
Low Stock Alert: USB-C Hub (Only 4 units left!)
```

Q:- 5. Update Product Information (Update Operation - UPDATE): 1. Update Quantity, 2. Update Price, 3. Confirm Updates.

- **Input:-**
- **step5_update_products.py:-**

```
import sqlite3

conn = sqlite3.connect('store.db')
cur = conn.cursor()

# 1. Update Quantity of HD Webcam
cur.execute('''
UPDATE products
SET quantity = 30
WHERE product_name = 'HD Webcam';
''')
print("1. Quantity of 'HD Webcam' updated to 30.")

# 2. Update Price of Wireless Mouse
cur.execute('''
UPDATE products
SET price = 499.00
WHERE product_name = 'Wireless Mouse';
''')
print("2. Price of 'Wireless Mouse' updated to Rs. 499.00.")

conn.commit()

# 3. Confirm Updates
print("\n--- 3. Confirmation of Updated Products ---")
cur.execute("SELECT product_name, price, quantity FROM products WHERE product_name
IN ('HD Webcam', 'Wireless Mouse');")
for r in cur.fetchall():
    print(f"Product: {r[0]:<15} | New Price: Rs.{r[1]} | New Quantity: {r[2]}")

conn.close()
```

- **Output:-**

```
1. Quantity of 'HD Webcam' updated to 30.
2. Price of 'Wireless Mouse' updated to Rs. 499.00.
--- 3. Confirmation of Updated Products ---
Product: Wireless Mouse  | New Price: Rs.499.0 | New Quantity: 25
Product: HD Webcam       | New Price: Rs.1800.0 | New Quantity: 30
```

Q:- 6. Remove Products (Delete Operation - DELETE): 1. Delete a Product, 2. Confirm Deletion.

- **Input:-**
- **step6_delete_product.py:-**

```
import sqlite3

conn = sqlite3.connect('store.db')
cur = conn.cursor()

# 1. Delete a Product (USB-C Hub)
cur.execute("DELETE FROM products WHERE product_name = 'USB-C Hub';")
conn.commit()
print("1. Product 'USB-C Hub' deleted from database.")

# 2. Confirm Deletion
print("\n--- 2. Inventory Check after Deletion ---")
cur.execute("SELECT product_id, product_name FROM products;")
remaining = cur.fetchall()
for p in remaining:
    print(f"ID: {p[0]} | Name: {p[1]}")

conn.close()
```

- **Output:-**

```
1. Product 'USB-C Hub' deleted from database.

--- 2. Inventory Check after Deletion ---
ID: 1 | Name: Wireless Mouse
ID: 2 | Name: Mechanical Keyboard
ID: 3 | Name: HD Webcam
ID: 5 | Name: Gaming Headset
```

Q:- 7. Understanding commit() and rollback(): 1. Demonstrate Commit, 2. Demonstrate Rollback.

- **Input:-**
- **step7_commit_and_rollback.py:-**

```
import sqlite3

conn = sqlite3.connect('store.db')
cur = conn.cursor()

# 1. Demonstrate Commit
print("--- 1. Demonstrating commit() ---")
cur.execute("INSERT INTO products (product_name, price, quantity) VALUES ('Bluetooth
Speaker', 1499.0, 10);")
conn.commit()  # Changes are permanently saved
print("Product 'Bluetooth Speaker' inserted and committed.")

cur.execute("SELECT product_name FROM products WHERE product_name = 'Bluetooth
Speaker';")
print("Found after commit:", cur.fetchone()[0])

# 2. Demonstrate Rollback
print("\n--- 2. Demonstrating rollback() ---")
cur.execute("INSERT INTO products (product_name, price, quantity) VALUES ('Accidental
Dummy Product', 99999.0, 0);")
print("Inserted 'Accidental Dummy Product' (uncommitted).")

# Rollback changes before commit
conn.rollback()
print("Transaction rolled back (changes reverted)!")

cur.execute("SELECT product_name FROM products WHERE product_name = 'Accidental
Dummy Product';")
result = cur.fetchone()
print("Searching for uncommitted product after rollback:", result)
print("Observation: The uncommitted record was completely removed by rollback().")

conn.close()
```

- **Output:-**

```
--- 1. Demonstrating commit() ---
Product 'Bluetooth Speaker' inserted and committed.
Found after commit: Bluetooth Speaker
```

--- 2. Demonstrating rollback() ---
Inserted 'Accidental Dummy Product' (uncommitted).
Transaction rolled back (changes reverted)!
Searching for uncommitted product after rollback: None
Observation: The uncommitted record was completely removed by rollback().

Q:- 8. Basic Database Backup (Dump): 1. Dump Database, 2. Save to File.

- **Input:-**
- **step8_dump_backup.py:-**

```
import sqlite3

conn = sqlite3.connect('store.db')

# 1. Dump Database & 2. Save to File
backup_filename = 'store_backup.sql'
with open(backup_filename, 'w') as f:
    for line in conn.iterdump():
        f.write(f"{line}\n")

print(f"Database successfully dumped and backed up to '{backup_filename}'.")

# Preview the backup file content
print("\n--- Backup File Preview (First 5 lines) ---")
with open(backup_filename, 'r') as f:
    for i in range(5):
        print(f.readline().strip())

conn.close()
```

- **Output:-**

```
Database successfully dumped and backed up to 'store_backup.sql'.

--- Backup File Preview (First 5 lines) ---
BEGIN TRANSACTION;
CREATE TABLE products (
    product_id INTEGER PRIMARY KEY AUTOINCREMENT,
    product_name TEXT NOT NULL,
    price REAL NOT NULL,
```

Q:- 9. Export Products to CSV File.

- **Input:-**
- **step9_export_csv.py:-**

```
import sqlite3
```

```
import csv

conn = sqlite3.connect('store.db')
cur = conn.cursor()

# Retrieve all products
cur.execute("SELECT * FROM products;")
rows = cur.fetchall()

csv_file = 'products_exported.csv'
with open(csv_file, 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['Product_ID', 'Product_Name', 'Price', 'Quantity'])
    writer.writerows(rows)

print(f"Products exported successfully to '{csv_file}'.")

# Print CSV content
print("\n--- Exported CSV Contents ---")
with open(csv_file, 'r') as f:
    print(f.read())

conn.close()
```

- **Output:-**

```
Products exported successfully to 'products_exported.csv'.

--- Exported CSV Contents ---
Product_ID,Product_Name,Price,Quantity
1,Wireless Mouse,499.0,25
2,Mechanical Keyboard,2200.0,15
3,HD Webcam,1800.0,30
5,Gaming Headset,2800.0,18
6,Bluetooth Speaker,1499.0,10
```

Q – 1 Create a Blank window with Tkinter library.

- **Input :-**

```
from tkinter import *

# Main window
root = Tk()

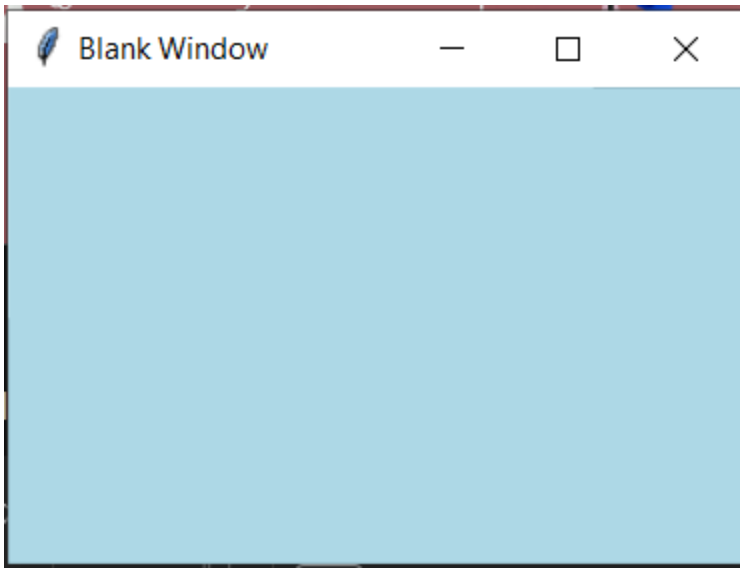
# Window title
root.title("Blank Window")

# Window size
root.geometry("400x400")

# Window background color
root.configure(bg="lightblue")

# Run the application
root.mainloop()
```

- **Output :-**



Q – 2 Create a window with the help of pack() method (add button widgets inside window).

- **Input :-**

```
from tkinter import *

# Main window
root = Tk()

# Window title
root.title("Button Widgets")

# Window size
root.geometry("400x400")

# Window background color
root.configure(bg="lightblue")

# Label
lbl = Label(
   root,
   text="Welcome to Tkinter",
   font=("Arial", 16, "bold"),
   bg="lightblue"
)
lbl.pack(pady=10)

# First button
btn1 = Button(
   root,
   text="Button 1",
   font=("Arial", 12),
   bg="green",
   fg="white"
)
btn1.pack(pady=10)
```

```
# Second button
btn2 = Button(
    root,
    text="Button 2",
    font=("Arial", 12),
    bg="blue",
    fg="white"
)
btn2.pack(pady=10)

# Third button
btn3 = Button(
    root,
    text="Button 3",
    font=("Arial", 12),
    bg="red",
    fg="white"
)
btn3.pack(pady=10)

# Run the application
root.mainloop()
```

- **Output :-**



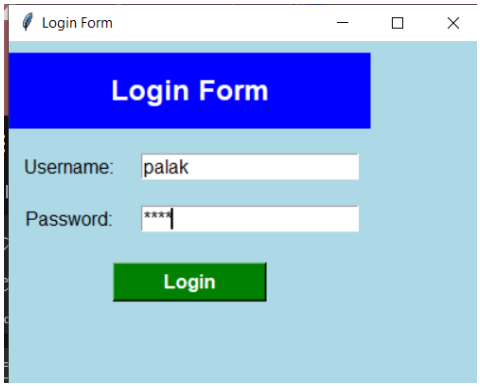
Q – 3 Write a code to create login form using grid() method.

- **Input :-**

```
from tkinter import *
from tkinter import messagebox
root = Tk()
root.title("Login Form")
root.geometry("400x300")
root.configure(bg="lightblue")
lbl = Label(
   root,
   text="Login Form",
   font=("Arial", 18, "bold"),
   fg="white",
   bg="blue",
   width=20,
   height=2
)
lbl.grid(row=0, column=0, columnspan=2, pady=10
username_lbl = Label(
   root,
   text="Username:",
   font=("Arial", 12),
   bg="lightblue"
)
username_lbl.grid(row=1, column=0, padx=10, pady=10)
username_entry = Entry(
   root,
   width=20,
   font=("Arial", 12)
)
username_entry.grid(row=1, column=1, padx=10, pady=10)
password_lbl = Label(
   root,
   text="Password:",
   font=("Arial", 12),
   bg="lightblue"
```

```
)
password_lbl.grid(row=2, column=0, padx=10, pady=10)
password_entry = Entry(
  root,
  width=20,
  font=("Arial", 12),
  show="*"
)
password_entry.grid(row=2, column=1, padx=10, pady=10)
def login():
  username = username_entry.get()
  password = password_entry.get()
  if username == "admin" and password == "1234":
    messagebox.showinfo("Success", "Login Successful")
  else:
    messagebox.showerror("Error", "Invalid Username or Password")
btn = Button(
  root,
  text="Login",
  command=login,
  font=("Arial", 12, "bold"),
  bg="green",
  fg="white",
  width=12
btn.grid(row=3, column=0, columnspan=2, pady=15)
root.mainloop()
```

- **Output :-**



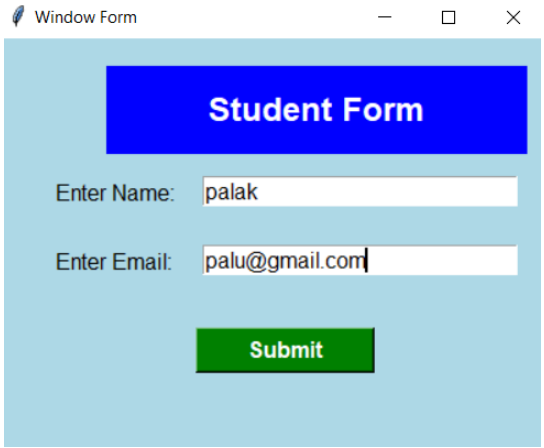
Q – 4 Create a window form with the help of place() method.

- **Input :-**

```
from tkinter import *
# Main window
root = Tk()
root.title("Window Form")
root.geometry("400x300")
root.configure(bg="lightblue")
# Heading label
lbl = Label(
   root,
   text="Student Form",
   font=("Arial", 18, "bold"),
   fg="white",
   bg="blue",
   width=20,
   height=2
)
lbl.place(x=80, y=20)
# Name label
name_lbl = Label(
   root,
   text="Enter Name:",
   font=("Arial", 12),
   bg="lightblue"
)
name_lbl.place(x=40, y=100)
# Name entry
name_entry = Entry(
   root,
   width=25,
   font=("Arial", 12)
)
name_entry.place(x=150, y=100)
# Email label
email_lbl = Label(
```

```
    root,
    text="Enter Email:",
    font=("Arial", 12),
    bg="lightblue"
)
email_lbl.place(x=40, y=150)
# Email entry
email_entry = Entry(
    root,
    width=25,
    font=("Arial", 12)
)
email_entry.place(x=150, y=150)
# Submit button
btn = Button(
    root,
    text="Submit",
    font=("Arial", 12, "bold"),
    bg="green",
    fg="white",
    width=12
)
btn.place(x=145, y=210)
# Run the application
root.mainloop()
```

- **Output :-**



Q – 5 Divide a window form into 2 parts using frame.

- **Input :-**

```
from tkinter import *

# Main Window
root = Tk()
root.title("Window Divided into 2 Parts")
root.geometry("500x350")
root.configure(bg="lightblue")

# Heading
heading = Label(
   root,
   text="Student Registration Form",
   font=("Arial", 18, "bold"),
   bg="lightblue",
   fg="darkblue"
)
heading.pack(pady=10)

# First Frame
frame1 = Frame(
   root,
   bg="white",
   bd=2,
   relief="groove",
   width=220,
   height=230
)
frame1.place(x=20, y=60)

# First Frame Heading
Label(
   frame1,
   text="Student Details",
   font=("Arial", 14, "bold"),
```

```
    bg="white",
    fg="blue"
).place(x=35, y=10)

# Name
Label(frame1, text="Name:", bg="white").place(x=10, y=60)
Entry(frame1, width=18).place(x=75, y=60)

# Roll Number
Label(frame1, text="Roll No:", bg="white").place(x=10, y=100)
Entry(frame1, width=18).place(x=75, y=100)

# Second Frame
frame2 = Frame(
    root,
    bg="lightyellow",
    bd=2,
    relief="groove",
    width=220,
    height=230
)
frame2.place(x=260, y=60)

# Second Frame Heading
Label(
    frame2,
    text="Contact Details",
    font=("Arial", 14, "bold"),
    bg="lightyellow",
    fg="darkgreen"
).place(x=30, y=10)

# Mobile Number
Label(frame2, text="Mobile:", bg="lightyellow").place(x=10, y=60)
Entry(frame2, width=18).place(x=75, y=60)

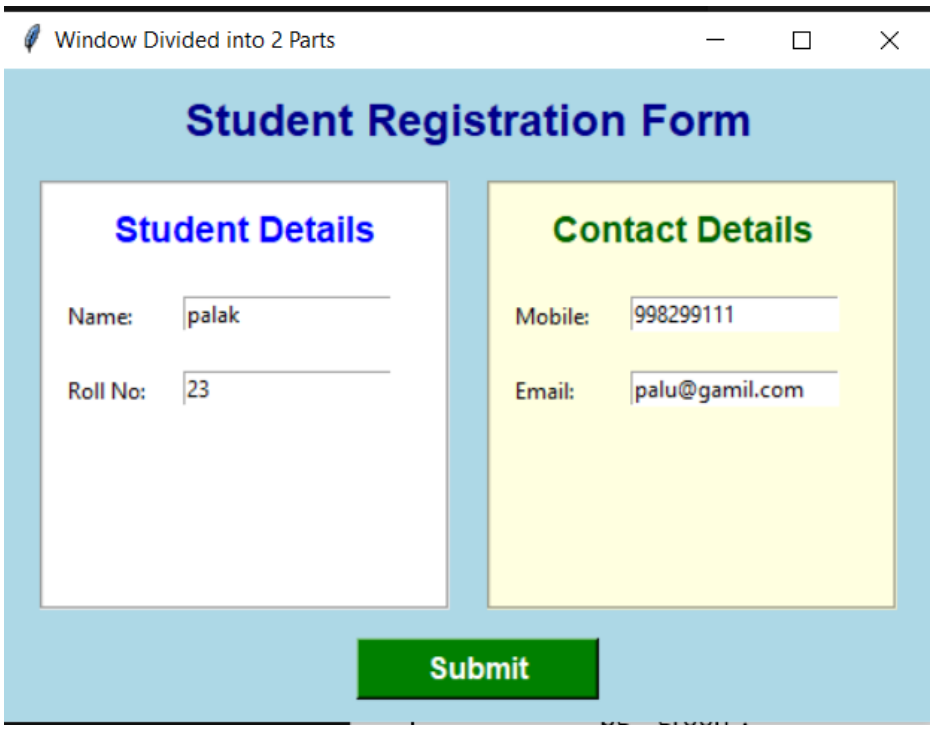
# Email
Label(frame2, text="Email:", bg="lightyellow").place(x=10, y=100)
```

```
Entry(frame2, width=18).place(x=75, y=100)

# Submit Button
Button(
   root,
   text="Submit",
   font=("Arial", 12, "bold"),
   bg="green",
   fg="white",
   width=12
).place(x=190, y=305)

# Run Window
root.mainloop()
```

➢ **Output :-**



Q – 7 Write a code to demonstrate Button Widget in Tkinter:

- **Input :-**

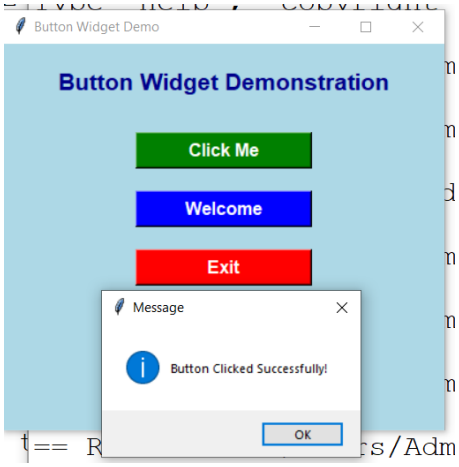
```
from tkinter import *
from tkinter import messagebox

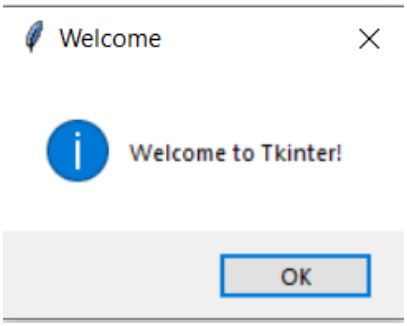
# Main Window
root = Tk()
root.title("Button Widget Demo")
root.geometry("400x350")
root.configure(bg="lightblue")

# Heading Label
heading = Label(
   root,
   text="Button Widget Demonstration",
   font=("Arial", 16, "bold"),
   bg="lightblue",
   fg="darkblue"
)
heading.pack(pady=20)
def click_button():
   messagebox.showinfo("Message", "Button Clicked Successfully!")
def welcome_button():
   messagebox.showinfo("Welcome", "Welcome to Tkinter!")
btn1 = Button(
   root,
   text="Click Me",
   font=("Arial", 12, "bold"),
   bg="green",
   fg="white",
   width=15,
   command=click_button
)
btn1.pack(pady=10)
# Second Button
btn2 = Button(
```

```
    root,
    text="Welcome",
    font=("Arial", 12, "bold"),
    bg="blue",
    fg="white",
    width=15,
    command=welcome_button
)
btn2.pack(pady=10)
btn3 = Button(
    root,
    text="Exit",
    font=("Arial", 12, "bold"),
    bg="red",
    fg="white",
    width=15,
    command=root.destroy
)
btn3.pack(pady=10)
root.mainloop()
```

➢ **Output :-**





Q – 8 Write a code to demonstrate Checkbutton Widget

- **Input :-**

```
from tkinter import *
from tkinter import messagebox

# Main Window
root = Tk()
root.title("Checkbutton Widget Demo")
root.geometry("400x400")
root.configure(bg="lightblue")

# Heading Label
heading = Label(
   root,
   text="Checkbutton Demonstration",
   font=("Arial", 16, "bold"),
   bg="lightblue",
   fg="darkblue"
)
heading.pack(pady=20)

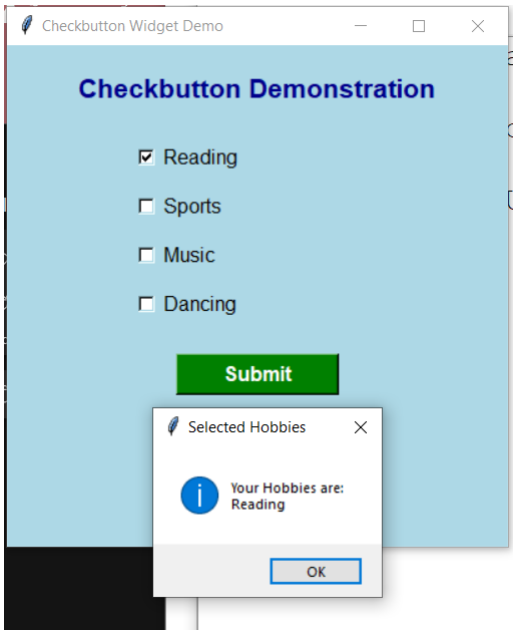
# Variables for Checkbuttons
reading_var = IntVar()
sports_var = IntVar()
music_var = IntVar()
dancing_var = IntVar()
# Reading Checkbutton
cb1 = Checkbutton(
   root,
   text="Reading",
   variable=reading_var,
   font=("Arial", 12),
   bg="lightblue"
)
cb1.pack(anchor="w", padx=100, pady=5)
# Sports Checkbutton
```

```
cb2 = Checkbutton(
    root,
    text="Sports",
    variable=sports_var,
    font=("Arial", 12),
    bg="lightblue"
)
cb2.pack(anchor="w", padx=100, pady=5)
# Music Checkbutton
cb3 = Checkbutton(
    root,
    text="Music",
    variable=music_var,
    font=("Arial", 12),
    bg="lightblue"
)
cb3.pack(anchor="w", padx=100, pady=5)
# Dancing Checkbutton
cb4 = Checkbutton(
    root,
    text="Dancing",
    variable=dancing_var,
    font=("Arial", 12),
    bg="lightblue"
)
cb4.pack(anchor="w", padx=100, pady=5)
def submit():
    hobbies = []
    if reading_var.get() == 1:
        hobbies.append("Reading")

    if sports_var.get() == 1:
        hobbies.append("Sports")
    if music_var.get() == 1:
        hobbies.append("Music")
    if dancing_var.get() == 1:
        hobbies.append("Dancing")
    if hobbies:
```

```
        messagebox.showinfo(
            "Selected Hobbies",
            "Your Hobbies are:\n" + "\n".join(hobbies)
        )
    else:
        messagebox.showwarning(
            "Warning",
            "Please select at least one hobby."
        )
btn = Button(
    root,
    text="Submit",
    font=("Arial", 12, "bold"),
    bg="green",
    fg="white",
    width=12,
    command=submit
)
btn.pack(pady=20)
root.mainloop()
```

- **Output :-**



Q – 9 Write a code to demonstrate Entry Widget

- **Input :-**

```
from tkinter import *
from tkinter import messagebox

# Main Window
root = Tk()
root.title("Entry Widget Demo")
root.geometry("450x350")
root.configure(bg="lightblue")

# Heading Label
heading = Label(
   root,
   text="Entry Widget Demonstration",
   font=("Arial", 16, "bold"),
   bg="lightblue",
   fg="darkblue"
)
heading.pack(pady=20)

# Name Label
name_label = Label(
   root,
   text="Enter Name:",
   font=("Arial", 12),
   bg="lightblue"
)
name_label.place(x=50, y=90)

# Name Entry
name_entry = Entry(root, width=25, font=("Arial", 12))
name_entry.place(x=170, y=90)

# Email Label
email_label = Label(
```

```
    root,
    text="Enter Email:",
    font=("Arial", 12),
    bg="lightblue"
)
email_label.place(x=50, y=140)

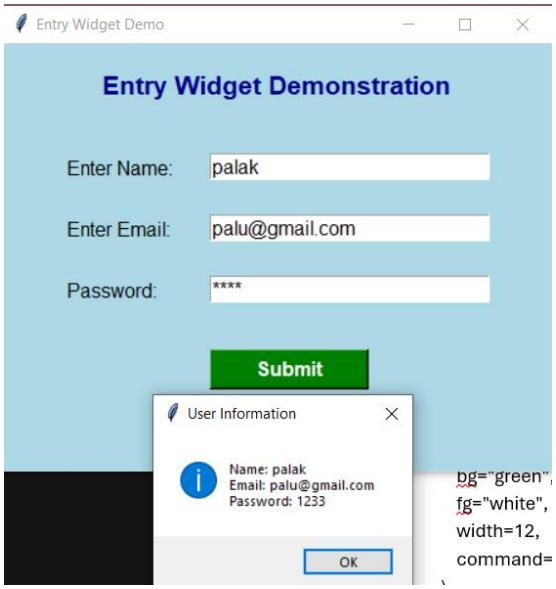
# Email Entry
email_entry = Entry(root, width=25, font=("Arial", 12))
email_entry.place(x=170, y=140)

# Password Label
password_label = Label(
    root,
    text="Password:",
    font=("Arial", 12),
    bg="lightblue"
)
password_label.place(x=50, y=190)

# Password Entry
password_entry = Entry(
    root,
    width=25,
    font=("Arial", 12),
    show="*"
)
password_entry.place(x=170, y=190)
# Submit Function
def submit():
    name = name_entry.get()
    email = email_entry.get()
    password = password_entry.get()
    if not name or not email or not password:
        messagebox.showwarning(
            "Warning",
            "Please fill in all fields."
        )
```

```
        return
    messagebox.showinfo(
        "User Information",
        f"Name: {name}\n"
        f"Email: {email}\n"
        f"Password: {password}"
    )
btn = Button(
    root,
    text="Submit",
    font=("Arial", 12, "bold"),
    bg="green",
    fg="white",
    width=12,
    command=submit
)
btn.place(x=170, y=250)
# Run Application
root.mainloop()
```

- **Output :-**



Q – 10 Write a code to demonstrate Frame Widget

- **Input :-**

```
from tkinter import *
from tkinter import messagebox

# Main Window
root = Tk()
root.title("Frame Widget Demo")
root.geometry("500x400")
root.configure(bg="lightblue")

# Heading Label
heading = Label(
   root,
   text="Frame Widget Demonstration",
   font=("Arial", 18, "bold"),
   bg="lightblue",
   fg="darkblue"
)
heading.pack(pady=15)

# First Frame - Student Details
frame1 = Frame(
   root,
   bg="white",
   bd=2,
   relief="groove",
   width=440,
   height=120
)
frame1.pack(padx=20, pady=10)
frame1.pack_propagate(False)

# First Frame Heading
Label(
   frame1,
```

```
    text="Student Details",
    font=("Arial", 14, "bold"),
    bg="white",
    fg="blue"
).grid(row=0, column=0, columnspan=2, pady=5)

# Name Label and Entry
Label(frame1, text="Name:", bg="white").grid(
    row=1, column=0, padx=10, pady=5
)

name_entry = Entry(frame1, width=25)
name_entry.grid(row=1, column=1, padx=10, pady=5)

# Roll Number Label and Entry
Label(frame1, text="Roll No:", bg="white").grid(
    row=2, column=0, padx=10, pady=5
)

roll_entry = Entry(frame1, width=25)
roll_entry.grid(row=2, column=1, padx=10, pady=5)

# Second Frame - Contact Details
frame2 = Frame(
    root,
    bg="lightyellow",
    bd=2,
    relief="groove",
    width=440,
    height=100
)
frame2.pack(padx=20, pady=10)
frame2.pack_propagate(False)

# Second Frame Heading
Label(
    frame2,
    text="Contact Details",
```

```
    font=("Arial", 14, "bold"),
    bg="lightyellow",
    fg="darkgreen"
).grid(row=0, column=0, columnspan=2, pady=5)

# Email Label and Entry
Label(frame2, text="Email:", bg="lightyellow").grid(
    row=1, column=0, padx=10, pady=5
)

email_entry = Entry(frame2, width=25)
email_entry.grid(row=1, column=1, padx=10, pady=5)

# Submit Function
def submit():
    name = name_entry.get()
    roll = roll_entry.get()
    email = email_entry.get()

    if not name or not roll or not email:
        messagebox.showwarning(
            "Warning",
            "Please fill in all fields."
        )
        return

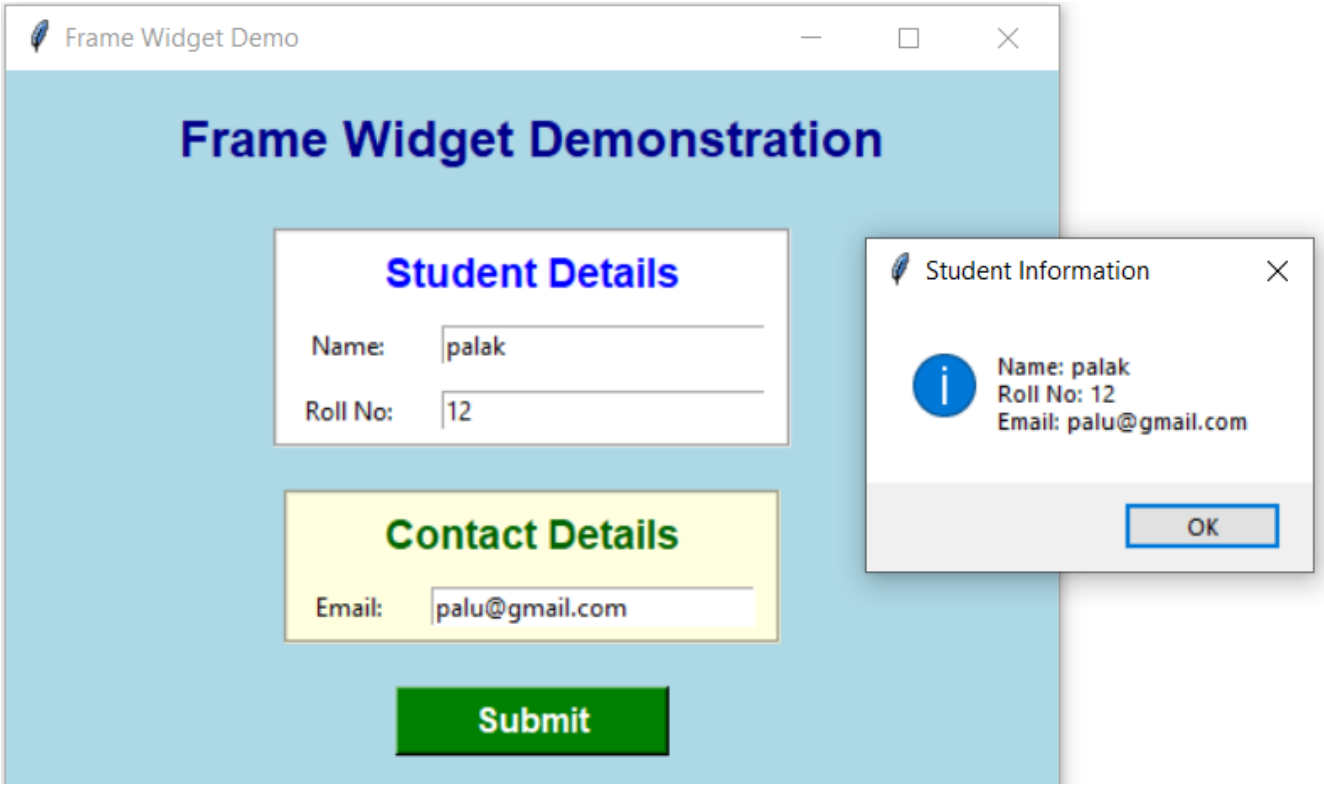
    messagebox.showinfo(
        "Student Information",
        f"Name: {name}\n"
        f"Roll No: {roll}\n"
        f"Email: {email}"
    )

# Submit Button
Button(
    root,
    text="Submit",
    font=("Arial", 12, "bold"),
```

```
    bg="green",
    fg="white",
    width=12,
    command=submit
).pack(pady=10)

# Run Application
root.mainloop()
```

➢ **Output :-**



Q – 11 Write a code to demonstrate Label Widget

- **Input :-**

```
from tkinter import *

# Main Window
root = Tk()
root.title("Label Widget Demo")
root.geometry("450x350")
root.configure(bg="lightblue")

# Heading Label
heading = Label(
   root,
   text="Label Widget Demonstration",
   font=("Arial", 18, "bold"),
   bg="lightblue",
   fg="darkblue"
)
heading.pack(pady=20)

# Simple Label
label1 = Label(
   root,
   text="Welcome to Python Tkinter",
   font=("Arial", 14),
   bg="white",
   fg="black"
)
label1.pack(pady=10)

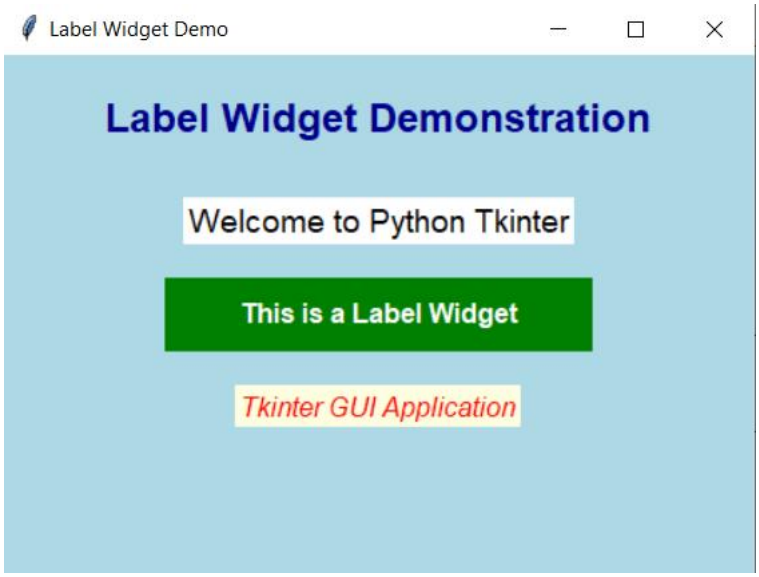
# Colored Label
label2 = Label(
   root,
   text="This is a Label Widget",
   font=("Arial", 12, "bold"),
   bg="green",
```

```
    fg="white",
    width=25,
    height=2
)
label2.pack(pady=10)

# Another Label
label3 = Label(
    root,
    text="Tkinter GUI Application",
    font=("Arial", 12, "italic"),
    bg="lightyellow",
    fg="red"
)
label3.pack(pady=10)

# Run Application
root.mainloop()
```

➢ **Output :-**



Q – 12 Write a code to demonstrate Listbox Widget

- **Input:-**

```
from tkinter import *
from tkinter import messagebox

# Main Window
root = Tk()
root.title("Listbox Widget Demo")
root.geometry("400x400")
root.configure(bg="lightblue")

# Heading Label
heading = Label(
   root,
   text="Listbox Widget Demonstration",
   font=("Arial", 16, "bold"),
   bg="lightblue",
   fg="darkblue"
)
heading.pack(pady=20)

# Subject Label
label = Label(
   root,
   text="Select Your Subject:",
   font=("Arial", 12),
   bg="lightblue"
)
label.pack(pady=5)

# Create Listbox
subject_list = Listbox(
   root,
   width=25,
   height=6,
   font=("Arial", 12),
```

```
    selectmode=SINGLE,
    exportselection=False
)
subject_list.pack(pady=10)

# Insert Items into Listbox
subjects = [
    "Python",
    "Java",
    "C Programming",
    "Web Development",
    "Database",
    "Computer Network"
]

for subject in subjects:
    subject_list.insert(END, subject)

# Submit Function
def submit():
    selected = subject_list.curselection()

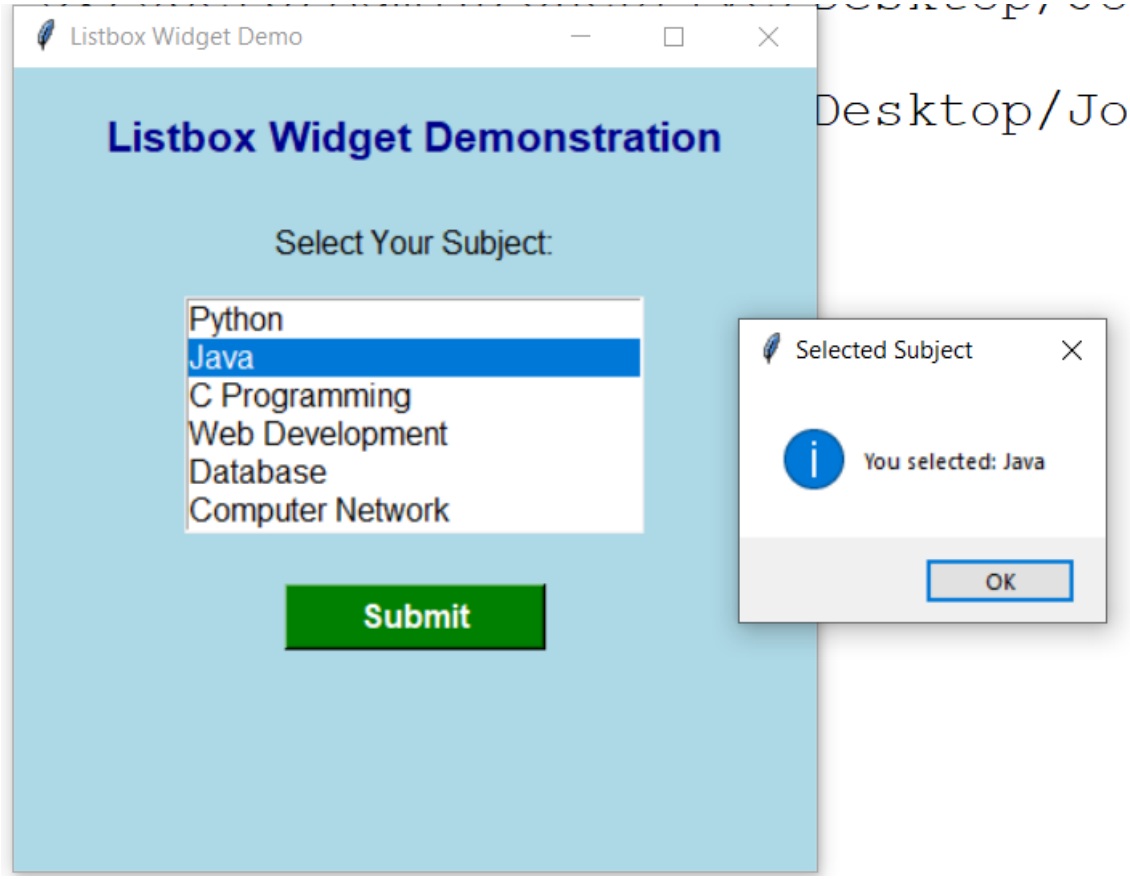
    if selected:
        subject = subject_list.get(selected[0])
        messagebox.showinfo(
            "Selected Subject",
            f"You selected: {subject}"
        )
    else:
        messagebox.showwarning(
            "Warning",
            "Please select a subject."
        )

# Submit Button
btn = Button(
    root,
    text="Submit",
```

```
    font=("Arial", 12, "bold"),
    bg="green",
    fg="white",
    width=12,
    command=submit
)
btn.pack(pady=15)

# Run Application
root.mainloop()
```

➢ **Output:-**



Q – 13 Write a code to demonstrate Radiobutton Widget in Tkinter.

- **Input:-**

```
from tkinter import *
from tkinter import messagebox

# Main Window
root = Tk()
root.title("Radiobutton Widget Demo")
root.geometry("400x350")
root.configure(bg="lightblue")

# Heading Label
heading = Label(
   root,
   text="Radiobutton Widget Demonstration",
   font=("Arial", 15, "bold"),
   bg="lightblue",
   fg="darkblue"
)
heading.pack(pady=20)

# Gender Label
label = Label(
   root,
   text="Select Your Gender:",
   font=("Arial", 12),
   bg="lightblue"
)
label.pack(pady=10)

# Variable for Radiobuttons
gender_var = StringVar(value="Male")

# Male Radiobutton
rb1 = Radiobutton(
   root,
   text="Male",
   variable=gender_var,
```

```
    value="Male",
    font=("Arial", 12),
    bg="lightblue"
)
rb1.pack(anchor="w", padx=130, pady=5)

# Female Radiobutton
rb2 = Radiobutton(
    root,
    text="Female",
    variable=gender_var,
    value="Female",
    font=("Arial", 12),
    bg="lightblue"
)
rb2.pack(anchor="w", padx=130, pady=5)

# Other Radiobutton
rb3 = Radiobutton(
    root,
    text="Other",
    variable=gender_var,
    value="Other",
    font=("Arial", 12),
    bg="lightblue"
)
rb3.pack(anchor="w", padx=130, pady=5)

# Submit Function
def submit():
    gender = gender_var.get()

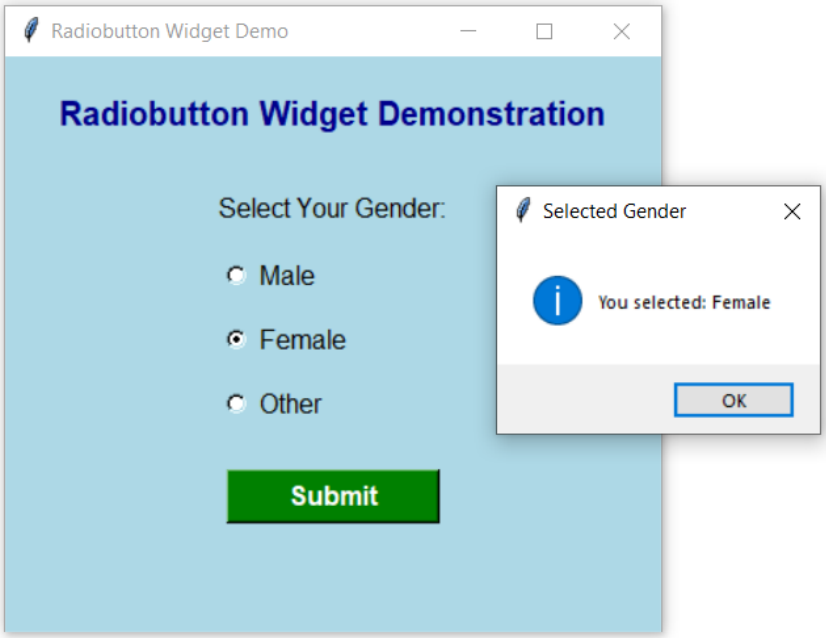
    messagebox.showinfo(
        "Selected Gender",
        f"You selected: {gender}"
    )

# Submit Button
```

```
btn = Button(
    root,
    text="Submit",
    font=("Arial", 12, "bold"),
    bg="green",
    fg="white",
    width=12,
    command=submit
)
btn.pack(pady=20)

# Run Application
root.mainloop()
```

➢ **Output:-**



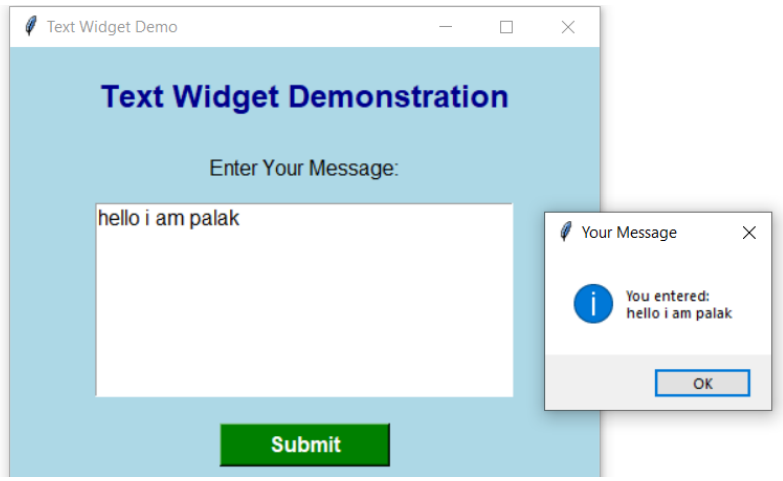
Q – 14 Write a code to demonstrate Text Widget in Tkinter.

- **Input:-**

```
from tkinter import *
from tkinter import messagebox
# Main Window
root = Tk()
root.title("Text Widget Demo")
root.geometry("450x400")
root.configure(bg="lightblue")
# Heading Label
heading = Label(
   root,
   text="Text Widget Demonstration",
   font=("Arial", 18, "bold"),
   bg="lightblue",
   fg="darkblue"
)
heading.pack(pady=20)
label = Label(
   root,
   text="Enter Your Message:",
   font=("Arial", 12),
   bg="lightblue"
)
label.pack(pady=5)
# Create Text Widget
text_box = Text(
   root,
   width=35,
   height=8,
   font=("Arial", 12),
   wrap=WORD
)
text_box.pack(pady=10)
# Submit Function
def submit():
   message = text_box.get("1.0", END).strip()
   if message:
```

```
        messagebox.showinfo(
            "Your Message",
            f"You entered:\n{message}"
        )
    else:
        messagebox.showwarning(
            "Warning",
            "Please enter your message."
        )
# Submit Button
btn = Button(
    root,
    text="Submit",
    font=("Arial", 12, "bold"),
    bg="green",
    fg="white",
    width=12,
    command=submit
)
btn.pack(pady=10)
# Run Application
root.mainloop()
```

➢ **Output:-**



Q – 15 Write a code to demonstrate Scale Widget in Tkinter.

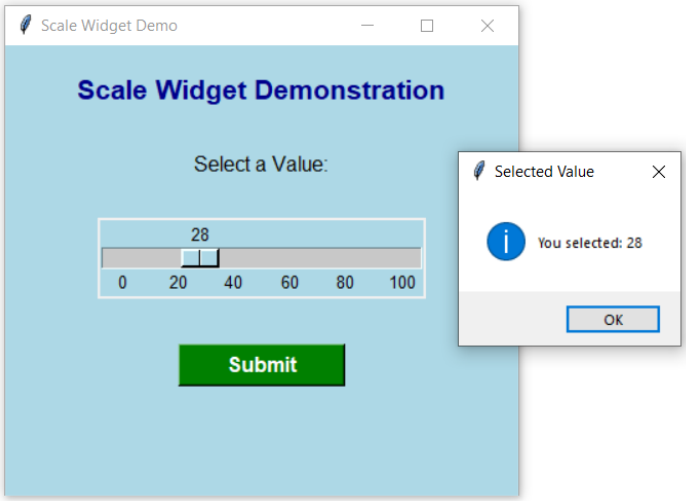
- **Input:-**

```
from tkinter import *
from tkinter import messagebox

# Main Window
root = Tk()
root.title("Scale Widget Demo")
root.geometry("400x350")
root.configure(bg="lightblue")
heading = Label(
   root,
   text="Scale Widget Demonstration",
   font=("Arial", 16, "bold"),
   bg="lightblue",
   fg="darkblue"
)
heading.pack(pady=20)
label = Label(
   root,
   text="Select a Value:",
   font=("Arial", 12),
   bg="lightblue"
)
label.pack(pady=10)
scale = Scale(
   root,
   from_=0,
   to=100,
   orient=HORIZONTAL,
   length=250,
   resolution=1,
   tickinterval=20,
   bg="lightblue",
   font=("Arial", 10)
)
scale.pack(pady=20)
# Submit Function
```

```
def submit():
    value = scale.get()
    messagebox.showinfo(
        "Selected Value",
        f"You selected: {value}"
    )
btn = Button(
    root,
    text="Submit",
    font=("Arial", 12, "bold"),
    bg="green",
    fg="white",
    width=12,
    command=submit
)
btn.pack(pady=15)
# Run Application
root.mainloop()
```

- **Output:-**



Q – 16 Write a code to demonstrate Toplevel Widget in Tkinter.

- **Input:-**

```
from tkinter import *
from tkinter import messagebox

root = Tk()
root.title("Main Window")
root.geometry("400x300")
root.config(bg="lightblue")

def open_window():
   new_window = Toplevel(root)
   new_window.title("New Window")
   new_window.geometry("300x200")
   new_window.config(bg="lightyellow")

   Label(
      new_window,
      text="Welcome to Toplevel Window!",
      font=("Arial", 12),
      bg="lightyellow"
   ).pack(pady=20)

   Button(
      new_window,
      text="Close Window",
      command=new_window.destroy
   ).pack(pady=10)

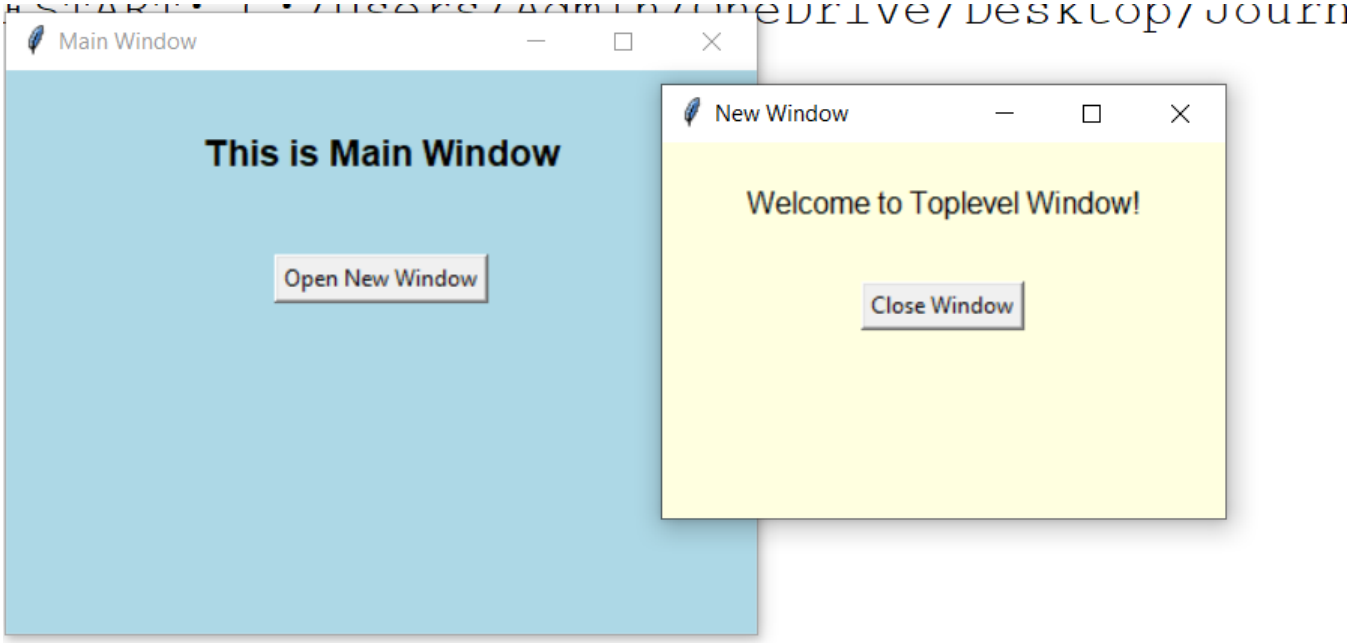
Label(
   root,
   text="This is Main Window",
   font=("Arial", 14, "bold"),
   bg="lightblue"
).pack(pady=30)

Button(
   root,
   text="Open New Window",
```

```
    command=open_window
).pack(pady=10)

root.mainloop()
```

- **Output:-**



Q – 17 Write a code to demonstrate simple calculations in Tkinter.

- **Input:-**

```
from tkinter import *
from tkinter import messagebox
root = Tk()
root.title("Simple Calculations")
root.geometry("400x400")
root.config(bg="lightblue")
def calculate(operation):
  try:
    num1 = float(entry1.get())
    num2 = float(entry2.get())
    if operation == "+":
      result = num1 + num2
    elif operation == "-":
      result = num1 - num2
    elif operation == "*":
      result = num1 * num2
    elif operation == "/":
      if num2 == 0:
        messagebox.showerror("Error", "Cannot divide by zero")
        return
      result = num1 / num2
    result_label.config(text="Result: " + str(result))
  except ValueError:
    messagebox.showerror(
      "Error", "Please enter valid numbers"
    )
Label(
  root,
  text="Simple Calculator",
  font=("Arial", 18, "bold"),
  bg="lightblue"
).pack(pady=15)
Label(root, text="Enter First Number:", bg="lightblue").pack()
entry1 = Entry(root)
entry1.pack(pady=5)
Label(root, text="Enter Second Number:", bg="lightblue").pack()
entry2 = Entry(root)
entry2.pack(pady=5)
```

```
Button(root, text="Addition (+)",
    command=lambda: calculate("+")).pack(pady=3)
Button(root, text="Subtraction (-)",
    command=lambda: calculate("-")).pack(pady=3)
Button(root, text="Multiplication (*)",
    command=lambda: calculate("*")).pack(pady=)
Button(root, text="Division (/)",
    command=lambda: calculate("/")).pack(pady=3)
result_label = Label(
  root,
  text="Result: ",
  font=("Arial", 12, "bold"),
  bg="lightblue"
)
result_label.pack(pady=10)
root.mainloop()
```

- **Output:-**

